Лабораторна робота **№ 2**

З дисципліни **«Розробка Інтернет клієнт-серверних систем»**

Виконав:

Студент 3 курсу

3 групи ІПЗ

Сідун Владислав Владиславович

2026

Тема: Керуючі конструкції та цикли в Go

Мета: Ознайомитися з основними способами роботи з керуючими

конструкціями та принципами роботи щ циклами в мові програмування Go.

Завдання: Навчитися використовувати керуючі конструкції і цикли, та вміти застосовувати їх при розробці простих програм.

Виконання

Варіант 11 : 5, 9, 11, 12, 17, 21, 27, 31, 35, 39

```
package main

import (
  "fmt"
  "math"
  "strconv"
  "strings"
)

func main() {
  fmt.Println("==================================================
")
  fmt.Println("       ЛАБОРАТОРНА РОБОТА №2 | ВАРІАНТ
11          ")
  fmt.Println("==================================================
")

  Task5()
  divider()

  Task9()
  divider()
```

```
  Task11()
  divider()

  Task12()
  divider()

  Task17()
  divider()

  Task21()
  divider()

  Task27()
  divider()

  Task31()
  divider()

  Task35()
  divider()

  Task39()

  fmt.Println("\n==================================================
==")
  fmt.Println("                ВИКОНАННЯ
ЗАВЕРШЕНО                 ")
  fmt.Println("==================================================
")
}

func divider() {
  fmt.Println("\n--------------------------------------------------
--")
}

// Завдання 5: Розрахунок ПДФО (2 версії коду)
```

```go
func Task5() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №5 (Умови)")
  salary := 60000.0
  fmt.Printf("Вхідні дані: Оклад = %.2f\n", salary)

  // Версія 1: Використання стандартного if-else
  var tax1 float64
  if salary > 50000.0 {
    tax1 = salary * 0.13
  } else {
    tax1 = salary * 0.06
  }

  // Версія 2: Використання switch без виразу (аналог ланцюга if-
else)
  var tax2 float64
  switch {
  case salary > 50000.0:
    tax2 = salary * 0.13
  default:
    tax2 = salary * 0.06
  }

  fmt.Printf("Результат (Версія if-else): Податок = %.2f\n",
tax1)
  fmt.Printf("Результат (Версія switch): Податок = %.2f\n", tax2)
}

// Завдання 9: Визначення чверті системи координат
func Task9() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №9 (Умови)")
  x, y := -4.5, 3.2
  fmt.Printf("Вхідні дані: x = %.1f, y = %.1f\n", x, y)

  fmt.Print("Результат: Точка знаходиться у ")
  if x > 0 && y > 0 {
    fmt.Println("1-й чверті")
```

```go
  } else if x < 0 && y > 0 {
    fmt.Println("2-й чверті")
  } else if x < 0 && y < 0 {
    fmt.Println("3-й чверті")
  } else if x > 0 && y < 0 {
    fmt.Println("4-й чверті")
  } else {
    fmt.Println("перетині осей (на осі або в центрі)")
  }
}

// Завдання 11: Перевірка належності точки колу (Радіус = 5,
Центр = 0,0)
func Task11() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №11 (Умови)")
  x, y := 3.0, 4.0
  radius := 5.0
  fmt.Printf("Вхідні дані: координати (%.1f, %.1f), радіус =
%.1f\n", x, y, radius)

  // Рівняння кола: x^2 + y^2 <= R^2
  if (x*x)+(y*y) <= (radius * radius) {
    fmt.Println("Результат: YES")
  } else {
    fmt.Println("Результат: NO")
  }
}

// Завдання 12: Перевірка, чи є введене значення числом
func Task12() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №12 (Умови)")
  input := "123.45" // Імітація вводу з клавіатури як рядка
  fmt.Printf("Вхідні дані: '%s'\n", input)

  // Спроба конвертувати рядок у число
  _, err := strconv.ParseFloat(input, 64)
```

```go
  if err == nil {
    fmt.Println("Результат: Number")
  } else {
    fmt.Println("Результат: Other")
  }
}

// Завдання 17: Конвертація температур (C <-> F)
func Task17() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №17 (Умови)")
  tempInput := "98.6f"
  fmt.Printf("Вхідні дані: '%s'\n", tempInput)

  tempInput = strings.ToLower(tempInput)
  lastChar := tempInput[len(tempInput)-1:]
  numStr := tempInput[:len(tempInput)-1]

  num, err := strconv.ParseFloat(numStr, 64)
  if err != nil {
    fmt.Println("Результат: Помилка формату числа")
    return
  }

  if lastChar == "c" {
    fahrenheit := (num * 9 / 5) + 32
    fmt.Printf("Результат: %.1fF\n", fahrenheit)
  } else if lastChar == "f" {
    celsius := (num - 32) * 5 / 9
    fmt.Printf("Результат: %.1fC\n", celsius)
  } else {
    fmt.Println("Результат: Невідомий формат вимірювання")
  }
}

// Завдання 21: Добуток елементів зрізу через цикл
func Task21() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №21 (Цикли)")
```

```go
  slice := []int{2, 3, 4, 5}
  fmt.Printf("Вхідні дані: %v\n", slice)

  product := 1
  for _, val := range slice {
    product *= val
  }
  fmt.Printf("Результат (добуток): %d\n", product)
}

// Завдання 27: Підрахунок входжень кожного символу в рядок
func Task27() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №27 (Цикли)")
  text := "hello world"
  fmt.Printf("Вхідні дані: '%s'\n", text)

  charCount := make(map[rune]int)
  for _, char := range text {
    charCount[char]++
  }

  fmt.Println("Результат:")
  for char, count := range charCount {
    fmt.Printf(" - '%c': %d разів\n", char, count)
  }
}

// Завдання 31: Сума математичного виразу з параметром z
func Task31() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №31 (Математичний цикл)")
  z := 3
  fmt.Printf("Вхідні дані: z = %d\n", z)

  sum := 0.0
  for n := 1; n <= z; n++ {
    nf := float64(n)
    inner := nf + 2.5*nf
```

```go
    numerator := math.Sqrt(math.Pow(inner, 3))
    sum += numerator / 4.0
  }
  fmt.Printf("Результат (сума): %.4f\n", sum)
}

// Завдання 35: Сума математичного виразу з параметром z
func Task35() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №35 (Математичний цикл)")
  z := 3
  fmt.Printf("Вхідні дані: z = %d\n", z)

  sum := 0.0
  for n := 1; n <= z; n++ {
    nf := float64(n)
    numerator := math.Tan(nf - 2.0*nf)
    denominator := math.Sqrt(10.0 + 0.6*nf)
    sum += numerator / denominator
  }
  fmt.Printf("Результат (сума): %.4f\n", sum)
}

// Завдання 39: Сума математичного виразу з параметром z
func Task39() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №39 (Математичний цикл)")
  z := 3
  fmt.Printf("Вхідні дані: z = %d\n", z)

  sum := 0.0
  for n := 1; n <= z; n++ {
    nf := float64(n)
    numerator := 2.0*math.Pow(nf, 2) - 4.0*nf + 10.0
    denominator := 2.0 * nf
    sum += numerator / denominator
  }
  fmt.Printf("Результат (сума): %.4f\n", sum)
}
```

```
==================================================
       ЛАБОРАТОРНА РОБОТА №2 | ВАРІАНТ 11
==================================================
---> ВИКОНАННЯ ЗАВДАННЯ №5 (Умови)
Вхідні дані: Оклад = 60000.00
Результат (Версія if-else): Податок = 7800.00
Результат (Версія switch): Податок = 7800.00

--------------------------------------------------
---> ВИКОНАННЯ ЗАВДАННЯ №9 (Умови)
Вхідні дані: x = -4.5, y = 3.2
Результат: Точка знаходиться у 2-й чверті

--------------------------------------------------
---> ВИКОНАННЯ ЗАВДАННЯ №11 (Умови)
Вхідні дані: координати (3.0, 4.0), радіус = 5.0
Результат: YES

--------------------------------------------------
---> ВИКОНАННЯ ЗАВДАННЯ №12 (Умови)
Вхідні дані: '123.45'
Результат: Number

--------------------------------------------------
---> ВИКОНАННЯ ЗАВДАННЯ №17 (Умови)
Вхідні дані: '98.6f'
Результат: 37.0C

--------------------------------------------------
---> ВИКОНАННЯ ЗАВДАННЯ №21 (Цикли)
Вхідні дані: [2 3 4 5]
Результат (добуток): 120
```

```
--------------------------------------------------
---> ВИКОНАННЯ ЗАВДАННЯ №27 (Цикли)
Вхідні дані: 'hello world'
Результат:
 - 'w': 1 разів
 - 'r': 1 разів
 - 'd': 1 разів
 - 'h': 1 разів
 - 'e': 1 разів
 - 'l': 3 разів
 - 'o': 2 разів
 - ' ': 1 разів

--------------------------------------------------
---> ВИКОНАННЯ ЗАВДАННЯ №31 (Математичний цикл)
Вхідні дані: z = 3
Результат (сума): 14.7730

--------------------------------------------------
---> ВИКОНАННЯ ЗАВДАННЯ №35 (Математичний цикл)
Вхідні дані: z = 3
Результат (сума): 0.2160

--------------------------------------------------
---> ВИКОНАННЯ ЗАВДАННЯ №39 (Математичний цикл)
Вхідні дані: z = 3
Результат (сума): 9.1667

==================================================
                ВИКОНАННЯ ЗАВЕРШЕНО
==================================================
```

Відповіді на контрольні запитання

1. Які основні керуючі конструкції підтримує мова Go? Мова Go підтримує умовні оператори (if, else), оператор багатовимірного вибору (switch) та єдину конструкцію для циклів (for).

2. У чому синтаксична особливість оператора if у Go порівняно з іншими мовами програмування? Синтаксична особливість полягає у відсутності обов'язкових круглих дужок навколо умови (наприклад, не if (x > 0), а if x > 0). Проте, фігурні дужки {} є строго обов'язковими, навіть якщо тіло умови складається лише з одного рядка коду.

3. Яку додаткову можливість надає конструкція if щодо оголошення змінної перед умовою? Go дозволяє ініціалізувати локальну змінну безпосередньо в блоці оператора if, розділяючи ініціалізацію та умову крапкою з комою (наприклад, if x := 10; x > 5). Область видимості такої змінної суворо обмежена цим блоком if та його секціями else.

4. У чому полягає особливість оператора switch у Go щодо використання break? На відміну від C-подібних мов, у Go відбувається автоматичний вихід із блоку switch після виконання спрацьованого блоку case. Явно писати break у кінці кожного кейсу не потрібно.

5. Чим відрізняється звичайний switch від конструкції switch без виразу? Звичайний switch порівнює конкретне значення з варіантами у case. Switch без виразу (switch {}) виконує роль аналога довгого ланцюга if-else. У цьому випадку кожен case має містити логічний вираз, який повертає true або false (наприклад, case x > 0:).

6. Які форми використання циклу for існують у Go?

У Go існує чотири основні форми циклу for:

1. Класичний цикл із лічильником (for i := 0; i < 5; i++).
2. Цикл лише з умовою (аналог while).
3. Нескінченний цикл без умов (for {}).
4. Цикл перебору ітерабельних структур for range.

7. Як реалізувати аналог циклу while у Go? У Go відсутнє ключове слово while. Його аналог реалізується за допомогою циклу for, якому передається виключно умова продовження.

8. Для чого використовується конструкція for range і з якими типами даних вона працює? Конструкція for range використовується для безпечної ітерації. Під час

кожної ітерації вона повертає індекс та копію значення елемента. Вона працює з масивами, зрізами (slices), рядками (strings), картами (maps) та каналами (channels).

9. Яке призначення операторів break та continue у циклах?

Оператор break здійснює негайне примусове переривання виконання всього циклу. Оператор continue перериває поточну ітерацію циклу, пропускаючи залишок коду в тілі, і одразу переходить до перевірки умови для наступної ітерації.

10. Як працюють логічні оператори &&, ||, ! в умовних виразах?

- && (Логічне І): Повертає true виключно якщо обидва операнди є істинними.
- || (Логічне АБО): Повертає true якщо хоча б один із операндів є істинним.
- ! (Логічне НІ): Інвертує значення булевого виразу (змінює true на false і навпаки).

Вони використовують механізм "короткого замикання": якщо для && перша умова false, друга навіть не перевіряється.